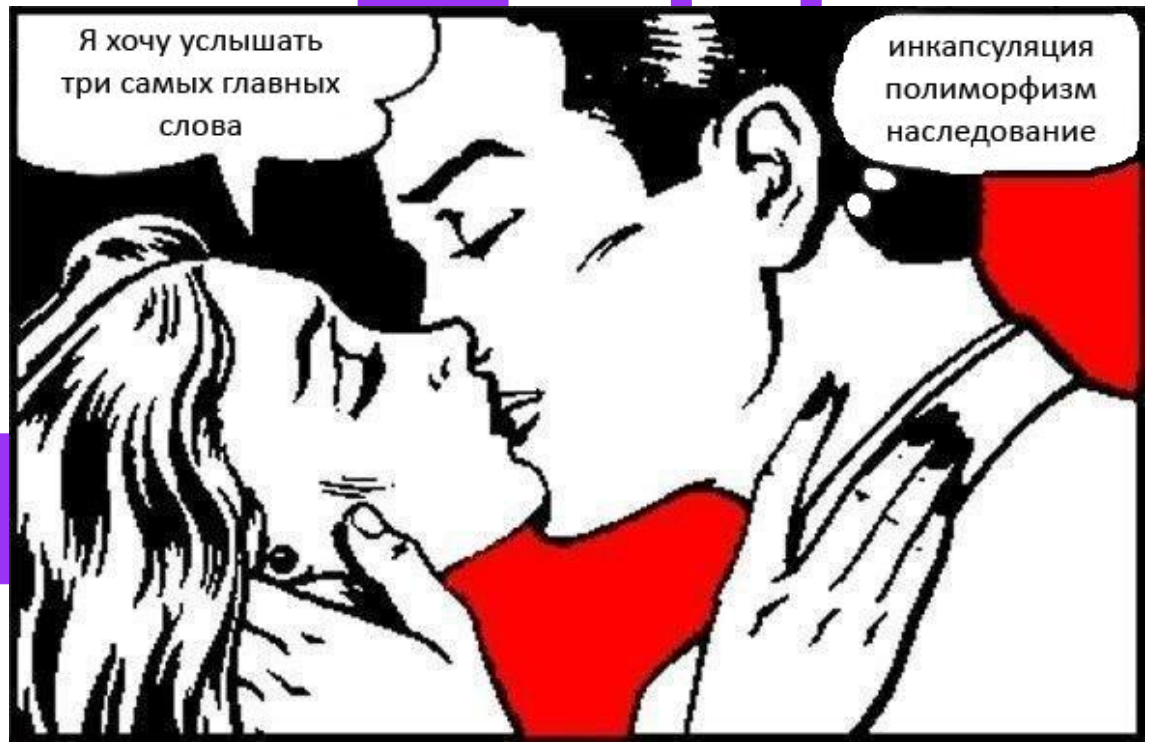
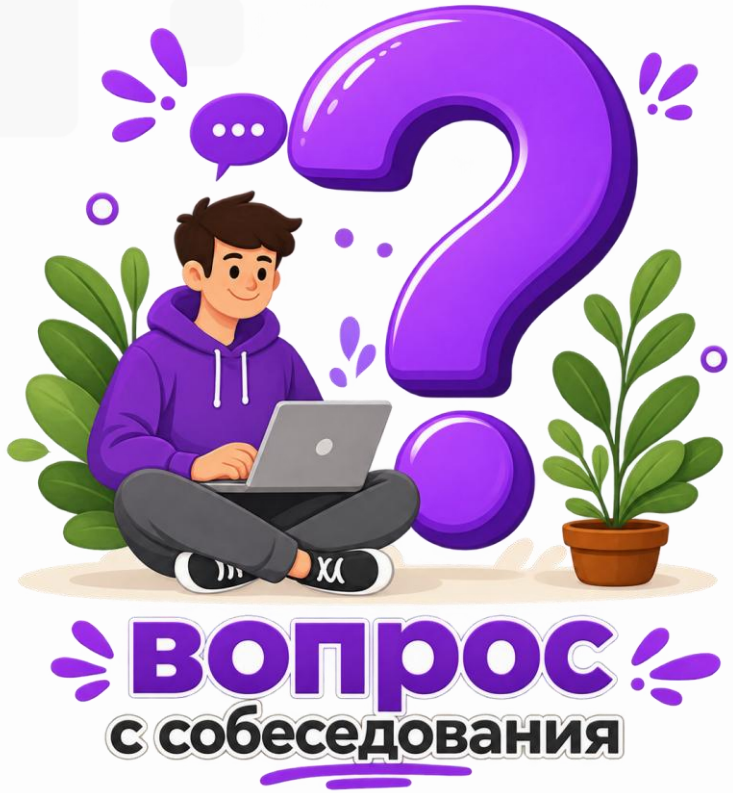
IThub

Понятие "Инкапсуляция", "Полиморфизм", "Наследование"



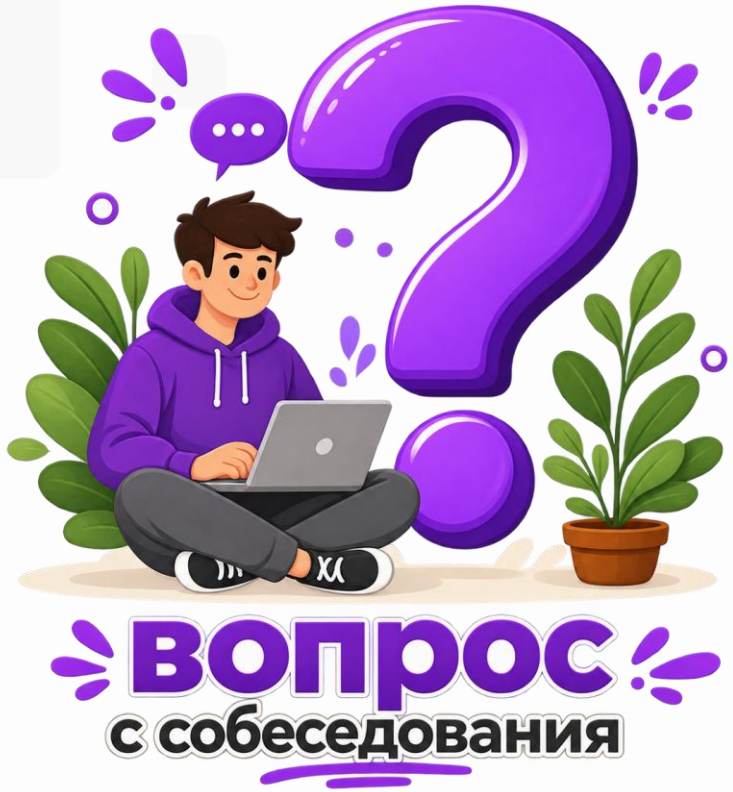


Что такое инкапсуляция?

Инкапсуляция - это свойство системы, позволяющее объединить данные и методы, работающие с ними, в классе и скрыть детали реализации от пользователя, открыв только то, что необходимо при последующем использовании.

Модификатор **private** делает доступными поля и методы класса только внутри данного класса. Это означает, что получить доступ к private полям из вне невозможно, как и нет возможности вызвать private методы.

IThub



Что такое полиморфизм?

Полиморфизм – это свойство системы использовать объекты с одинаковым интерфейсом без информации о типе и внутренней структуре объекта.

Преимуществом полиморфизма является то, что он помогает снижать сложность программ, разрешая использование одного и того же интерфейса для задания единого набора действий.

IThub

🔒 Инкапсуляция — «Моя личная жизнь закрыта от всех»

У нас есть телефон:

```
class Phone {
    private String password;

    public void unlock(String code) {
        if (code.equals(password)) {
            System.out.println("Телефон разблокирован");
        }
    }
}
```

Пароль спрятан внутри телефона (private).
Другие люди не могут сделать: phone.password = "1234";
Они могут только использовать кнопку входа (если знают пароль):
phone.unlock("1234");

👨‍👩‍👧 Наследование — «Дети получают способности родителей»

Пример: Семья блогеров

```
class Blogger {
    void createContent() {
        System.out.println("Создаю контент");
    }
}
```

```
class Gamer extends Blogger {
    void playGames() {
        System.out.println("Стримлю Minecraft");
    }
}
```

```
class TikToker extends Blogger {
    void dance() {
        System.out.println("Снимаю танцы");
    }
}
```

Родитель дал детям базовые способности

Полиморфизм — «Одна команда, разные реакции»

Пример: голосовое сообщение

```
class Person {
    void sayHello() {
        System.out.println("Привет");
    }
}
```

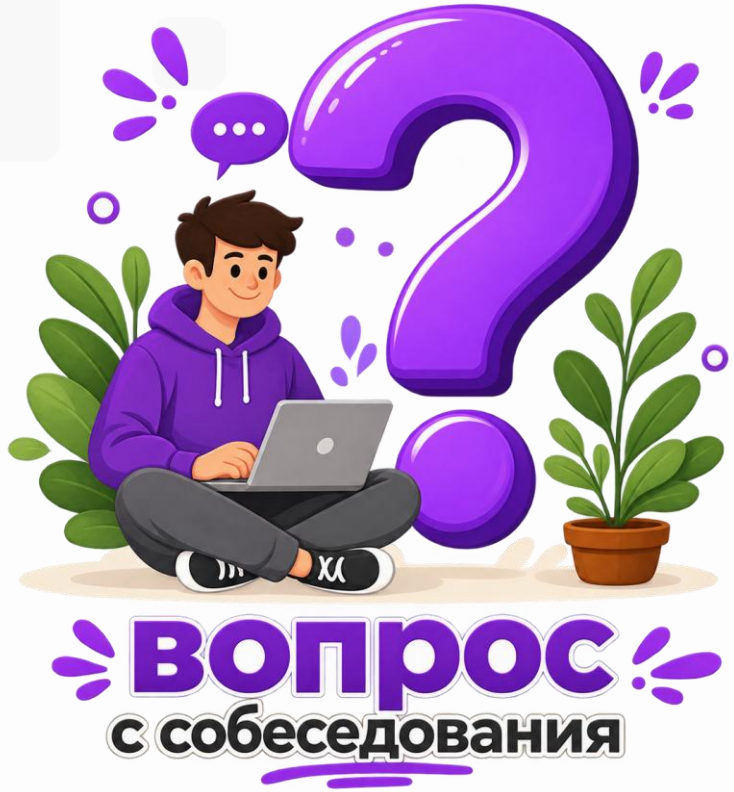
```
class Student extends Person {
    void sayHello() {
        System.out.println("Йо, как дела?");
    }
}
```

```
class Teacher extends Person {
    void sayHello() {
        System.out.println("Здравствуйте!");
    }
}
```

```
Person p1 = new Student();
Person p2 = new Teacher();

p1.sayHello();
p2.sayHello();
```

Йо, как дела?
Здравствуйте!



Что такое класс Object?

Object это базовый класс для всех остальных объектов в Java. **Любой класс наследуется от Object** и, соответственно, наследуют его методы.

IThub

Наследование

- позволяет создавать иерархические классификации
- унаследованный класс называется **суперклассом**, а класс, выполняющий наследование – **подклассом**
- ключевое слово extends
- хотя подкласс включает в себя все члены своего суперкласса, он не может получить доступ к тем членам суперкласса, которые были объявлены как закрытые (private)
- когда подклассу необходимо сослаться на свой непосредственный суперкласс, он может воспользоваться ключевым словом super
- в Java класс может наследоваться **только от одного** родительского класса
- позволяет использовать уже написанный код, что уменьшает дублирование
- когда нужно предотвратить наследование класса, перед определением класса указывается ключевое слово final
- все классы в Java являются подклассами Object

IThub

```java
public class Animal {

    private String color;
    private int age;
    private int weight;

    public Animal(String color, int age, int weight) {
        this.color = color;
        this.age = age;
        this.weight = weight;
    }

     // ... getters and setters …

    public void eat() {
        System.out.println("This animal eats food.");
    }
}
```

```java
public class Dog extends Animal {

    private String name;
    private String breed;

    public Dog(String color, int age, int weight, String name,
                    String breed) {
        super(color, age, weight);
        this.name = name;
        this.breed = breed;
    }

    // ... getters and setters …

    public void bark() {
        System.out.println("The dog barks");
    }
}
```

IThub

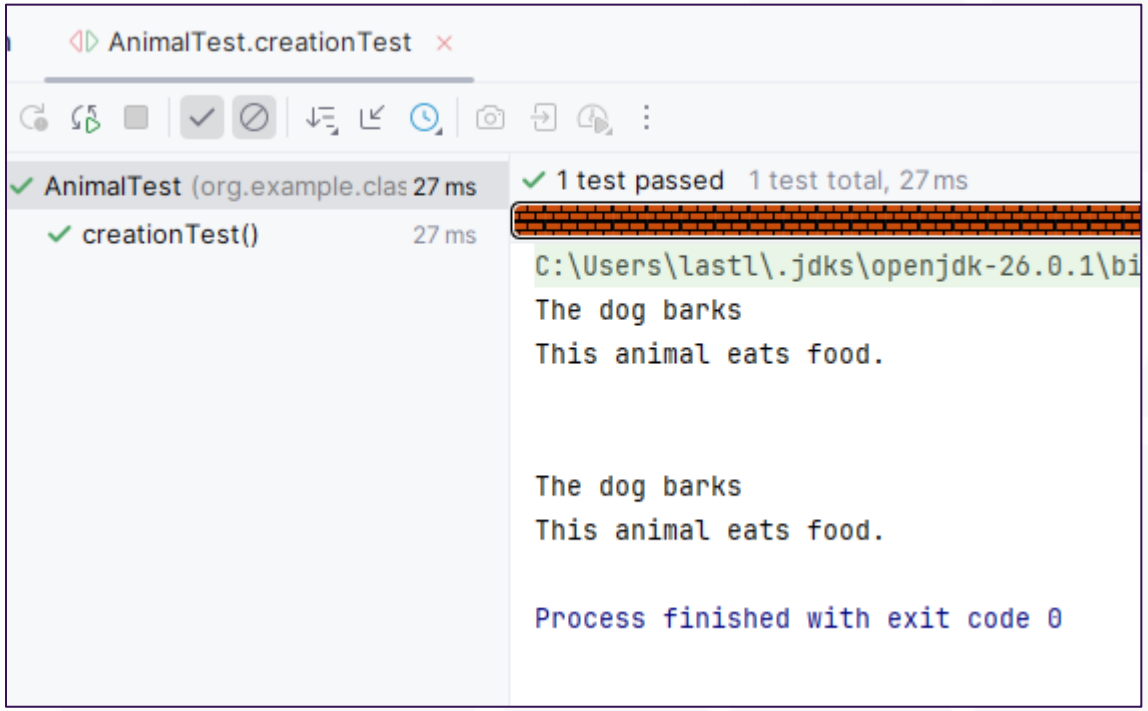
```java
@Test
public void creationTest() {
    Dog dog = new Dog();
    dog.bark();
    dog.eat();

    System.out.println("\n");

    Dog dogOllie = new Dog("gray", 5, 15, "Ollie", "Collie");
    dogOllie.bark();
    dogOllie.eat();

    Assertions.assertEquals(5, dogOllie.getAge());
    Assertions.assertEquals("Collie", dogOllie.getBreed());
    Assertions.assertEquals("Ollie", dogOllie.getName());
    Assertions.assertEquals("gray", dogOllie.getColor());
    Assertions.assertEquals(15, dogOllie.getWeight());
}
```

Методы доступа (геттеры и сеттеры) реализуют принцип **инкапсуляции**



IThub

Задание

В приложении банка у пользователя есть банковский счёт.

Баланс счёта нельзя изменить напрямую. Чтобы пополнить деньги или снять их, пользователь должен воспользоваться специальными действиями в приложении.

Какой принцип ООП используется?

Задание

Есть общий класс «Транспорт».

От него создаются классы:

- Автомобиль
- Самолёт
- Корабль

Все они имеют общие свойства (скорость, название), но каждый добавляет свои особенности.

Какой принцип ООП используется?

Задание

В игре есть команда: «Атаковать»

Но разные персонажи выполняют её по-разному:

- Воин атакует мечом
- Маг использует заклинание
- Лучник стреляет из лука

Какой принцип ООП используется?

Задание

В телефоне пользователя пароль скрыт.

Нельзя просто открыть настройки и изменить значение пароля напрямую.
Доступ к нему происходит через проверку кода.

Какой принцип ООП используется?

Задание

Есть класс «Животное».

От него создаются:

- Собака
- Кошка
- Птица

Все животные имеют имя и возраст, но у каждого появляются свои дополнительные характеристики.

Какой принцип ООП используется?